Gene Set Enrichment Analysis (GSEA): Whole Body Hyperthermia (WBH) Treatment

Victoria Chen, Anisha Kolambe, Ethan Aidam

Presented Monday, May 11

Contents

Overview

This analysis starts from the STAR-derived raw gene count matrix (`wbh_counts_matrix.csv`) and walks through quality filtering, differential expression with DESeq2, and a custom implementation of Gene Set Enrichment Analysis (GSEA). We test four biologically motivated gene sets, heat shock/proteostasis, interferon signaling, serotonin, and dopamine, for coordinated expression changes between Whole Body Hyperthermia (WBH)-treated and sham-treated samples. Afterwards, GSEA will be conducted using the original desktop software developed by the Broad Institute, the standard pipeline in biological research.

Biological Context

This dataset contains RNA-bulk sequencing data from 19 whole blood samples of individuals with Major Depressive Disorder (MDD), collected 30 minutes after Whole Body Hyperthermia (WBH) treatment or sham treatment. This data is referenced from Akonom et. al's paper (https://doi.org/10.1016/j.bbih.2026.101225), with more specific information regarding individual sample metadata found at (https://www.ncbi.nlm.nih.gov/Traces/study/?acc=PRJNA1287674&o=acc_s%3Aa). The original study aimed to compare the whole-blood transcriptomic response in hopes of identifying how biological pathways relevant to MDD may have been affected post-WBH, especially since WBH individuals exhibited sustained antidepressant effects without the side effects typically posed by common antidepressant drugs. We hope to recapitulate the upregulation in heat shock and immune pathways observed in the pathway, while also investigating if there are enrichment in molecular mechanism pathways related to neurotransmitters such as dopamine and serotonin (that current pharmacotherapies typically act on).

Load libraries

```r
library(AnnotationDbi)
library(org.Hs.eg.db)
library(GO.db)
library(edgeR)
library(DESeq2)
library(tidyverse)
library(gt)
```

Load raw STAR count matrix

Column names are cleaned of BAM and tab-file suffixes added by STAR (RNA-seq alignment software), and Ensembl gene IDs are mapped to HGNC (gene id) symbols using `org.Hs.eg.db` to match the notation typically used in GSEA gene sets for better compatability. Genes with no matching symbol retain their Ensembl ID, and all row names are made unique before downstream use to avoid errors and resolve possible multimapping (possibility that Ensembl IDs may map to the same gene symbol due to transcriptional variants or overlapping loci.).

```r
# Load the counts matrix
cnt <- read.csv(
  "wbh_counts_matrix.csv",
  row.names = 1,
  check.names = FALSE
)
cnt <- as.matrix(cnt)

# Clean sample names if STAR/BAM suffixes are present.
colnames(cnt) <- basename(colnames(cnt))
colnames(cnt) <- sub("\\.Aligned\\.sortedByCoord\\.out\\.bam$", "", colnames(cnt))
colnames(cnt) <- sub("\\.ReadsPerGene\\.out\\.tab$", "", colnames(cnt))

# Remove version numbers if present (e.g., ENSG... .1)
ensembl_ids <- sub("\\..*$", "", rownames(cnt))

# Map Ensembl IDs to gene symbols using the standard Bioconductor annotation package for human data
gene_annot <- AnnotationDbi::select(
  org.Hs.eg.db,
  keys = ensembl_ids,
  columns = c("SYMBOL"),
  keytype = "ENSEMBL"
)

# Match annotation back to count matrix
gene_symbols <- gene_annot$SYMBOL[match(ensembl_ids, gene_annot$ENSEMBL)]

# Replace missing symbols with Ensembl IDs
gene_symbols[is.na(gene_symbols)] <- ensembl_ids[is.na(gene_symbols)]

# Ensure uniqueness (required for DESeq2)
gene_symbols <- make.unique(gene_symbols)
```

```r
rownames(cnt) <- gene_symbols

# Sanity checks
if (any(is.na(cnt))) {
  stop("NA values detected in count matrix.")
}
if (any(cnt < 0)) {
  stop("Negative values detected. DESeq2 requires non-negative raw counts.")
}

# Inspect dimensions and first columns
dim(cnt)
```

```
## [1] 78691    19
```

```r
head(cnt[, 1:min(4, ncol(cnt))])
```

```
##                 SRR34413163 SRR34413164 SRR34413165 SRR34413166
## DDX11L16                  0           0           0           3
## ENSG00000223972           0           0           0           0
## WASH7P                   53          29          19          11
## ENSG00000227232           0           0           0           0
## MIR6859-1                 0           0           0           0
## ENSG00000243485           3           4           5           4
```

Define Sham versus WBH groups

Each of the 19 samples are assigned to either the WBH group (10 samples) or the Sham group (9 samples) using a manually curated sample table keyed by SRR accession. `Sham` is set as the reference level throughout. A design matrix with an intercept and a `groupWBH` indicator is created for modeling. The intercept represents the baseline level of expression (control) while the group indicator models the change caused by the experimental group using a variable (0 for sham, 1 for WBH) to represent the difference between the reference and WBH groups.

```r
# Create a metadata table that shows which group each sample belongs to, set Sham as reference group
sample_info <- tibble(
  sample = c(
    "SRR34413163", "SRR34413164", "SRR34413165", "SRR34413166", "SRR34413167",
    "SRR34413168", "SRR34413169", "SRR34413170", "SRR34413171", "SRR34413172",
    "SRR34413174", "SRR34413175", "SRR34413176", "SRR34413177",
    "SRR34413178", "SRR34413179", "SRR34413180", "SRR34413181", "SRR34413182"
  ),
  group = c(
    rep("WBH", 10),
    rep("Sham", 9)
  )
) %>%
  mutate(group = factor(group, levels = c("Sham", "WBH"))) %>%
  as.data.frame()

rownames(sample_info) <- sample_info$sample

# Match count columns to sample metadata.
missing_samples <- setdiff(sample_info$sample, colnames(cnt))
extra_samples   <- setdiff(colnames(cnt), sample_info$sample)
```

```r
# Safety check to ensure that metadata and gene count matches
if (length(missing_samples) > 0) {
  stop(paste("Samples in sample_info but missing from cnt:",
             paste(missing_samples, collapse = ", ")))
}
if (length(extra_samples) > 0) {
  message("Samples in cnt but not sample_info will be dropped: ",
          paste(extra_samples, collapse = ", "))
}

cnt <- cnt[, sample_info$sample]

# create a group level vector
group  <- sample_info$group

# create design matrix
design <- model.matrix(~ group)

group
```

```
##  [1] WBH  WBH  WBH  WBH  WBH  WBH  WBH  WBH  WBH  WBH  Sham Sham Sham Sham Sham
## [16] Sham Sham Sham Sham
## Levels: Sham WBH
```

```r
design
```

```
##    (Intercept) groupWBH
## 1            1        1
## 2            1        1
## 3            1        1
## 4            1        1
## 5            1        1
## 6            1        1
## 7            1        1
## 8            1        1
## 9            1        1
## 10           1        1
## 11           1        0
## 12           1        0
## 13           1        0
## 14           1        0
## 15           1        0
## 16           1        0
## 17           1        0
## 18           1        0
## 19           1        0
## attr(,"assign")
## [1] 0 1
## attr(,"contrasts")
## attr(,"contrasts")$group
## [1] "contr.treatment"
```

Filter genes by CPM

Lowly expressed genes are removed by retaining only those with a count-per-million (CPM) greater than 1 in at least 2 samples. This step eliminates genes that are essentially absent from the data, since they will contribute more noise than true signal in analysis. This also increases computational testing efficiency and decreases the multiple testing burden (thereby increasing the statistical power to find meaningful differences in remaining genes) A `DGEList` object is then constructed and TMM normalization factors are computed.

```
# Filter genes by cpm
cpm_vals     <- cpm(cnt)
keep         <- rowSums(cpm_vals > 1) >= 2
cnt_filtered <- cnt[keep, ]

# Output filtering results
filter_summary <- tibble(
  genes_before_filtering = nrow(cnt),
  genes_after_filtering  = nrow(cnt_filtered),
  genes_removed          = nrow(cnt) - nrow(cnt_filtered)
)
filter_summary
```

```
## # A tibble: 1 x 3
##   genes_before_filtering genes_after_filtering genes_removed
##                    <int>                 <int>         <int>
## 1                  78691                 25422         53269
```

```
# Store counts and phenotype labels together
dge <- DGEList(
  counts  = cnt_filtered,
  group   = sample_info$group,
  samples = sample_info
)

# Add library sizes and normalization factors
dge <- calcNormFactors(dge)

# Confirm that phenotype labels are stored
dge$samples
```

```
##             group lib.size norm.factors      sample group.1
## SRR34413163   WBH 16335114    1.1235093 SRR34413163     WBH
## SRR34413164   WBH 19804376    0.9208956 SRR34413164     WBH
## SRR34413165   WBH 15374096    1.0706350 SRR34413165     WBH
## SRR34413166   WBH  8878932    1.5650826 SRR34413166     WBH
## SRR34413167   WBH 14379486    0.7819804 SRR34413167     WBH
## SRR34413168   WBH 15811439    1.0181677 SRR34413168     WBH
## SRR34413169   WBH 13107961    0.8028434 SRR34413169     WBH
## SRR34413170   WBH 16466302    1.0409032 SRR34413170     WBH
## SRR34413171   WBH 16202886    0.9948958 SRR34413171     WBH
## SRR34413172   WBH 16100634    1.0074875 SRR34413172     WBH
## SRR34413174  Sham 14400708    1.1965086 SRR34413174    Sham
## SRR34413175  Sham 16732529    0.9654859 SRR34413175    Sham
## SRR34413176  Sham 15296004    0.9503973 SRR34413176    Sham
## SRR34413177  Sham 17257518    0.9549282 SRR34413177    Sham
## SRR34413178  Sham 16627135    1.0604389 SRR34413178    Sham
## SRR34413179  Sham 15043451    1.0610420 SRR34413179    Sham
```

```
## SRR34413180  Sham 17403930    0.9380073 SRR34413180    Sham
## SRR34413181  Sham 14386025    0.8961813 SRR34413181    Sham
## SRR34413182  Sham 14048204    0.8721729 SRR34413182    Sham
```

Run DESeq2

DESeq2 is used to fit a negative-binomial model (accounts for overdispersion typically seen in RNA-seq data) for each gene with `group` as the sole covariate, contrasting WBH against Sham.

```
# Create dds object
dds <- DESeqDataSetFromMatrix(
  countData = cnt_filtered,
  colData   = sample_info,
  design    = ~ group
)

# Ensure Sham is the reference level.
dds$group <- relevel(dds$group, ref = "Sham")
dds       <- DESeq(dds)

# Identify how many genes were up/down regulated
res <- results(
  dds,
  contrast = c("group", "WBH", "Sham")
)

summary(res)
```

```
##
## out of 25403 with nonzero total read count
## adjusted p-value < 0.1
## LFC > 0 (up)       : 1, 0.0039%
## LFC < 0 (down)     : 3, 0.012%
## outliers [1]       : 0, 0%
## low counts [2]     : 19, 0.075%
## (mean count < 0)
## [1] see 'cooksCutoff' argument of ?results
## [2] see 'independentFiltering' argument of ?results
```

```
# Create a table and list by adjusted p-value
res_df <- as.data.frame(res) %>%
  rownames_to_column("gene_id") %>%
  arrange(padj)

# Top significant genes
head(res_df)
```

```
##           gene_id   baseMean log2FoldChange      lfcSE      stat       pvalue
## 1           DNM1L  738.35487     -0.2731949 0.05587220 -4.889640 1.010207e-06
## 2           ATG2B 1749.20307     -0.3105544 0.06317638 -4.915673 8.847793e-07
## 3           FKBP4  384.14476      0.7758856 0.16547921  4.688719 2.749202e-06
## 4         GRAMD1C  267.73423     -0.5692414 0.13179513 -4.319138 1.566395e-05
## 5            NCDN  237.75139      0.3571055 0.08446227  4.227989 2.357897e-05
## 6 ENSG00000241860   63.78633      0.7203787 0.17339053  4.154660 3.257715e-05
##         padj
## 1 0.01283115
```

```
## 2 0.01283115
## 3 0.02327933
## 4 0.09947783
## 5 0.11979531
## 6 0.13792624
```

Create ranked gene list for GSEA by log2 fold change

GSEA requires a gene list ranked by a metric that captures the magnitude of differential expression. Here we use the DESeq2 log2 fold change (WBH vs. Sham), sorted in decreasing order so that the most upregulated genes appear at the top of the list.

```
# Make the ranked gene list by removing genes where log2fc couldn't be calculated, keeping only gene na
gsea_ranked_log2fc <- res_df %>%
  dplyr::filter(!is.na(log2FoldChange)) %>%
  dplyr::select(gene_id, log2FoldChange) %>%
  dplyr::arrange(desc(log2FoldChange))

head(gsea_ranked_log2fc)
```

```
##           gene_id log2FoldChange
## 1       LINC00958       4.759068
## 2        TBC1D3.1       3.936580
## 3           RAB17       3.200490
## 4 ENSG00000295037       3.027509
## 5       LINC00355       3.017234
## 6         CYP2B7P       2.986241
```

Define gene sets for GSEA from GO IDs

Gene sets are assembled by querying `org.Hs.eg.db` for Biological Process GO terms whose descriptions match four keywords of interest: *heat/chaperone/protein folding/proteostasis*, *interferon*, *serotonin*, and *dopamine*. Only genes present in the ranked list are retained, giving four gene sets of sizes ranging from 46 (serotonin) to 374 (interferon signaling).

```
# Create named ranked vector for GSEA (gene -> statistic)
ranked_stats        <- gsea_ranked_log2fc$log2FoldChange
names(ranked_stats) <- gsea_ranked_log2fc$gene_id
ranked_stats        <- sort(ranked_stats, decreasing = TRUE)

# Map gene symbols to GO terms (Biological Process only)
go_map <- AnnotationDbi::select(
  org.Hs.eg.db,
  keys    = keys(org.Hs.eg.db, keytype = "SYMBOL"),
  columns = c("SYMBOL", "GO", "ONTOLOGY"),
  keytype = "SYMBOL"
) %>%
  filter(!is.na(GO), ONTOLOGY == "BP")

# Retrieve GO term names (Biological Process ontology)
go_terms <- AnnotationDbi::select(
  GO.db,
  keys    = keys(GO.db),
  columns = c("TERM", "ONTOLOGY"),
  keytype = "GOID"
```

```r
) %>%
  dplyr::filter(ONTOLOGY == "BP")

# Create mapping: GO ID -> gene symbols
go2gene <- split(go_map$SYMBOL, go_map$GO)

# Identify relevant GO terms by keyword matching
heat_go_ids <- go_terms %>%
  filter(str_detect(
    TERM,
    regex("heat|chaperone|protein folding|protein refolding|proteostasis",
          ignore_case = TRUE)
  )) %>%
  pull(GOID)

interferon_go_ids <- go_terms %>%
  filter(str_detect(TERM, regex("interferon", ignore_case = TRUE))) %>%
  pull(GOID)

serotonin_go_ids <- go_terms %>%
  filter(str_detect(TERM, regex("serotonin", ignore_case = TRUE))) %>%
  pull(GOID)

dopamine_go_ids <- go_terms %>%
  filter(str_detect(TERM, regex("dopamine", ignore_case = TRUE))) %>%
  pull(GOID)

# Build gene sets from selected GO terms
gene_sets <- list(
  heat_shock_proteostasis = unique(unlist(go2gene[heat_go_ids])),
  interferon_signaling    = unique(unlist(go2gene[interferon_go_ids])),
  serotonin               = unique(unlist(go2gene[serotonin_go_ids])),
  dopamine                = unique(unlist(go2gene[dopamine_go_ids]))
)

# Keep only genes present in the ranked list
gene_sets <- lapply(gene_sets, function(gs) {
  intersect(unique(gs), names(ranked_stats))
})

# Summarize gene set sizes after filtering
gene_set_summary <- tibble(
  gene_set               = names(gene_sets),
  n_genes_in_ranked_list = sapply(gene_sets, length)
)
gene_set_summary
```

```
## # A tibble: 4 x 2
##   gene_set                n_genes_in_ranked_list
##   <chr>                                    <int>
## 1 heat_shock_proteostasis                    293
## 2 interferon_signaling                       374
## 3 serotonin                                   46
## 4 dopamine                                   131
```

Define Enrichment Score function

The enrichment score (ES) is computed using the weighted Kolmogorov–Smirnov-like statistic from the original Subramanian et al. (2005) paper. At each position in the ranked gene list, the running sum increases by the gene's absolute fold-change (weighted hit) if the gene belongs to the set, and decreases by an equal penalty otherwise. The ES is the maximum absolute deviation of this running sum from zero.

```r
# Calculate enrichment scores
gsea_es <- function(ranked_stats, gene_set, p = 1) {
  genes <- names(ranked_stats)
  N     <- length(genes)
  hits  <- genes %in% gene_set
  Nh    <- sum(hits)

  if (Nh == 0) return(NA)

  # Compute weights
  weights <- abs(ranked_stats)^p

  # Normalization constants
  NR <- sum(weights[hits])

  # Initialize cumulative vectors
  Phit  <- numeric(N)
  Pmiss <- numeric(N)

  # Compute cumulative sums explicitly
  hit_cum  <- 0
  miss_cum <- 0

  for (i in seq_len(N)) {
    if (hits[i]) {
      hit_cum  <- hit_cum  + weights[i] / NR
    } else {
      miss_cum <- miss_cum + 1 / (N - Nh)
    }
    Phit[i]  <- hit_cum
    Pmiss[i] <- miss_cum
  }

  running_score <- Phit - Pmiss

  # Enrichment score = max absolute deviation
  ES <- running_score[which.max(abs(running_score))]

  return(list(
    ES            = ES,
    running_score = running_score,
    Phit          = Phit,
    Pmiss         = Pmiss
  ))
}
```

Permutation Test for Normalized Enrichment Score (NES) and P-value

Statistical significance is assessed via phenotype permutation. Rather than refitting the full DESeq2 model 1,000 times (which would be prohibitively slow), we permute sample labels and recompute a fast ranking statistic: the difference in mean log-CPM between the permuted WBH and Sham groups. The observed ES for each gene set is then normalized by the mean of the same-sign null ES values to produce the NES, and an empirical p-value is derived from the fraction of permutations with a null |ES| greater than or equal to the observed |ES|. Multiple testing correction uses both Benjamini–Hochberg (BH) and Storey's q-value.

```r
# phenotype permutation using logCPM mean differences
# prior.count = 1 adds a small constant to prevent log(0) and dampen noise
logcpm <- edgeR::cpm(dge, log = TRUE, prior.count = 1)

# Rank genes based on group differences (log2fc equivalent), genes are sorted by most upregulated to do
rank_from_labels <- function(logcpm, group) {
  group <- factor(group, levels = c("Sham", "WBH"))
  stat  <- rowMeans(logcpm[, group == "WBH",  drop = FALSE]) -
           rowMeans(logcpm[, group == "Sham", drop = FALSE])
  sort(stat, decreasing = TRUE)
}

# Calculate Normalized Enrichment Score (adjusts for size of the gene set and the null distribution)
compute_NES <- function(observed_ES, null_dist) {
  if (is.na(observed_ES)) return(NA_real_)
  if (observed_ES >= 0) {
    pos_null <- null_dist[null_dist >= 0]
    if (length(pos_null) == 0 || mean(pos_null) == 0) return(NA_real_)
    observed_ES / mean(pos_null)
  } else {
    neg_null <- null_dist[null_dist < 0]
    if (length(neg_null) == 0 || mean(abs(neg_null)) == 0) return(NA_real_)
    observed_ES / mean(abs(neg_null))
  }
}

# Run GSEA function using Phenotype Permutation
gsea_pheno_perm <- function(dge, gene_sets, observed_ranked_stats, nperm = 1000) {
  logcpm <- edgeR::cpm(dge, log = TRUE, prior.count = 1)

  # Calculate the actual observed Enrichment Scores for all gene sets
  observed_ES <- sapply(gene_sets, function(gs) {
    gsea_es(observed_ranked_stats, gs)$ES
  })

  # Store ES results from random permutations
  null_mat <- matrix(
    NA_real_,
    nrow     = nperm,
    ncol     = length(gene_sets),
    dimnames = list(NULL, names(gene_sets))
  )

  # Shuffle labels randomly and re-calculate ES to build a "null" model
  for (b in seq_len(nperm)) {
    perm_group        <- sample(dge$samples$group)
```

```r
    perm_ranked_stats  <- rank_from_labels(logcpm, perm_group)
    null_mat[b, ]      <- sapply(gene_sets, function(gs) {
      gsea_es(perm_ranked_stats, gs)$ES
    })
  }

  # Find empirical p-values
  pvalues <- sapply(seq_along(gene_sets), function(j) {
    mean(abs(null_mat[, j]) >= abs(observed_ES[j]), na.rm = TRUE)
  })

  # Calculate NES for each gene set to allow for comparison between pathways
  NES <- sapply(seq_along(gene_sets), function(j) {
    compute_NES(
      observed_ES = observed_ES[j],
      null_dist   = null_mat[, j]
    )
  })

  # Compile results
  gsea_df <- tibble(
    gene_set = names(gene_sets),
    ES       = observed_ES,
    NES      = NES,
    pvalue   = pvalues,
    padj_BH  = p.adjust(pvalues, method = "BH")
  ) %>%
    arrange(pvalue)

  # Estimate pi0 (proportion of truly null hypotheses) via Storey's method
  lambda    <- 0.5
  pi0_storey <- min(1, mean(gsea_df$pvalue > lambda, na.rm = TRUE) / (1 - lambda))

  # Add q-values (Storey) for better FDR control
  gsea_df <- gsea_df %>%
    mutate(
      pi0_storey    = pi0_storey,
      qvalue_storey = pi0_storey * pvalue
    )

  return(gsea_df)
}

# Run analysis
set.seed(114) # Using seeds allows for reproducibility of results
gsea_df <- gsea_pheno_perm(
  dge                   = dge,
  gene_sets             = gene_sets,
  observed_ranked_stats = ranked_stats,
  nperm                 = 1000
)
```

GSEA Results

None of the four gene sets reach statistical significance after multiple-testing correction (all BH-adjusted p-values >= 0.844). The heat shock/proteostasis set shows the most positive NES (1.103), consistent with the expected cellular response to elevated temperature, while the dopamine, serotonin, and interferon sets all have negative NES values, suggesting slight depletion relative to chance.

These results are likely because we used phenotype permutations. Since we have a limited sample set (only 19 individuals), the number of permutations that are possible is much fewer. Therefore, if the biological signal is subtle, identification may be lost since random shuffles can produce enrichment scores that are equivalently high to the observed score purely by chance, inflating the raw p-values. Our ranked structure also lacks weighting (for example, the paper used sign(LFC)x-log10(p) to ensure that genes were both highly changed and statistically confident were the primary drivers of the Enrichment score.

```r
# Show results
gene_set_labels <- c(
  heat_shock_proteostasis = "Heat Shock Proteasis",
  dopamine                = "Dopamine",
  serotonin               = "Serotonin",
  interferon_signaling    = "Interferon Signaling"
)

gsea_df <- gsea_df %>%
  mutate(
    gene_set = recode(gene_set, !!!gene_set_labels)
  )

gsea_table <- gsea_df |>
  gt() |>
  tab_header(
    title    = "Gene Set Enrichment Analysis Results",
    subtitle = "Summary of enrichment scores and significance metrics"
  ) |>
  cols_label(
    gene_set      = "Gene Set",
    ES            = "ES",
    NES           = "NES",
    pvalue        = "P-value",
    padj_BH       = "Adj. P (BH)",
    pi0_storey    = "Pi0 (Storey)",
    qvalue_storey = "Q-value"
  ) |>
  fmt_number(columns = c(ES, NES), decimals = 3) |>
  fmt_number(columns = c(pvalue, padj_BH, qvalue_storey), decimals = 3) |>
  tab_style(
    style     = cell_text(weight = "bold"),
    locations = cells_column_labels(everything())
  ) |>
  cols_align(align = "center", columns = -gene_set) |>
  tab_options(
    table.font.size     = 10,
    table.width         = pct(100),
    data_row.padding    = px(3),
    column_labels.padding = px(3)
  )
```

Gene Set Enrichment Analysis Results

Summary of enrichment scores and significance metrics

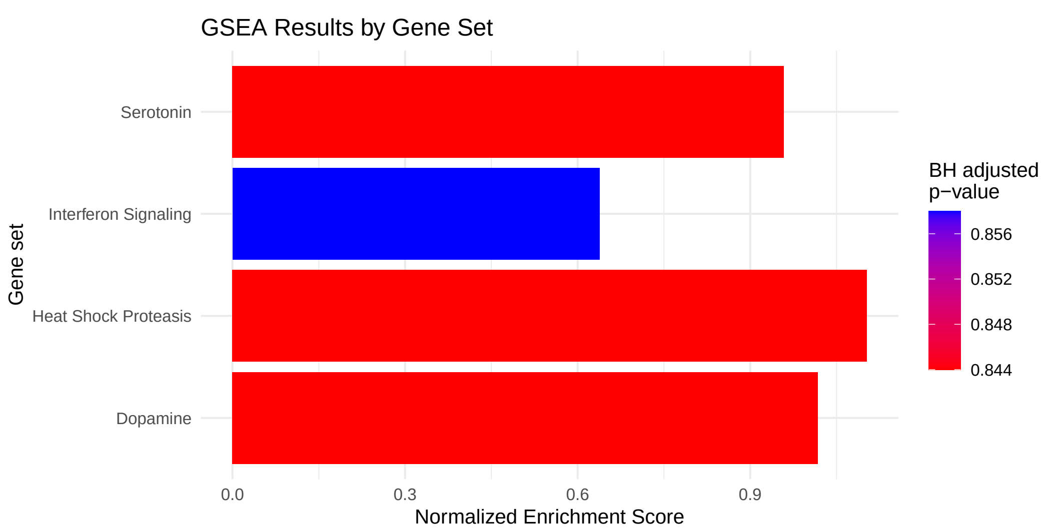
Gene Set	ES	NES	P-value	Adj. P (BH)	Pi0 (Storey)	Q-value
Heat Shock Proteasis	0.215	1.103	0.307	0.844	1	0.307
Dopamine	-0.541	-1.017	0.527	0.844	1	0.527
Serotonin	-0.463	-0.958	0.633	0.844	1	0.633
Interferon Signaling	-0.153	-0.638	0.858	0.858	1	0.858

```
gsea_table
```

Plotting Enrichment Scores

The bar chart below displays the absolute NES for each gene set, colored by its BH-adjusted p-value. Redder bars indicate lower (more significant) adjusted p-values, while bluer bars indicate higher (less significant) values.

```
# Create bar chart to visualize GSEA results
ggplot(gsea_df, aes(x = gene_set, y = abs(NES), fill = padj_BH)) +
  geom_col() +
  coord_flip() +
  scale_fill_gradient(
    low  = "red",
    high = "blue",
    name = "BH adjusted\np-value"
  ) +
  labs(
    title = "GSEA Results by Gene Set",
    x     = "Gene set",
    y     = "Normalized Enrichment Score"
  ) +
  theme_minimal()
```



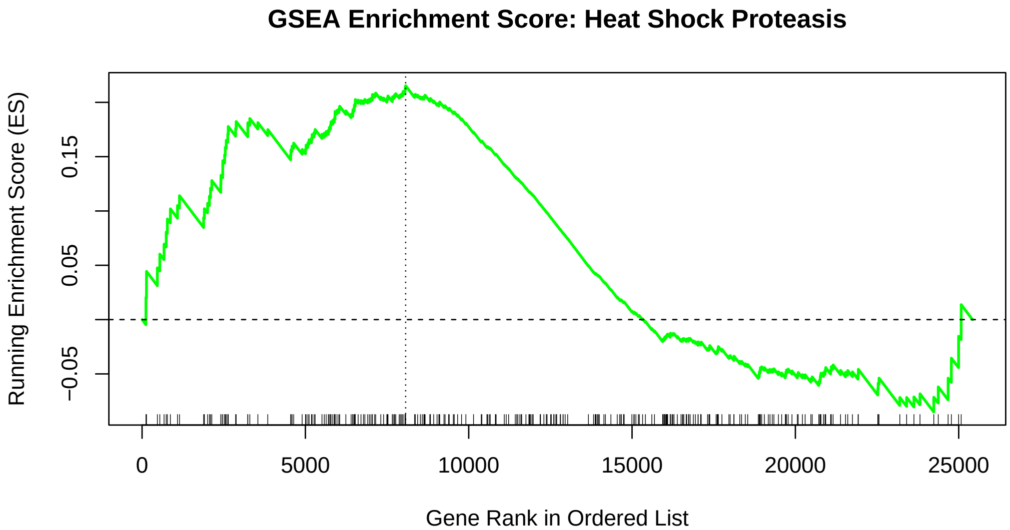
The running enrichment score plots below show, for each gene set, how the cumulative sum evolves as we walk down the ranked gene list from most upregulated (left) to most downregulated (right). Tick marks on the x-axis indicate where gene-set members appear in the list; the dotted vertical line marks the position of the maximum absolute deviation (the reported ES).

```r
# Create GSEA enrichment plot
plot_gsea_es <- function(ranked_stats, gene_set, gene_set_name) {
  es <- gsea_es(ranked_stats, gene_set)

  plot(
    es$running_score,
    type = "l",
    lwd  = 2,
    col  = "green",
    xlab = "Gene Rank in Ordered List",
    ylab = "Running Enrichment Score (ES)",
    main = paste("GSEA Enrichment Score:", gene_set_name)
  )
  abline(h = 0, lty = 2)

  hit_positions <- which(names(ranked_stats) %in% gene_set)
  rug(hit_positions)

  abline(
    v   = which.max(abs(es$running_score)),
    lty = 3
  )
}
```

Heat Shock / Proteostasis

The heat shock gene set (293 genes) shows a broad positive enrichment: the running score climbs steadily through the upper half of the ranked list before falling back, with its peak (ES = 0.215) occurring around rank 8,000. This pattern is consistent with mild but diffuse upregulation of chaperones and proteostasis genes following WBH treatment.

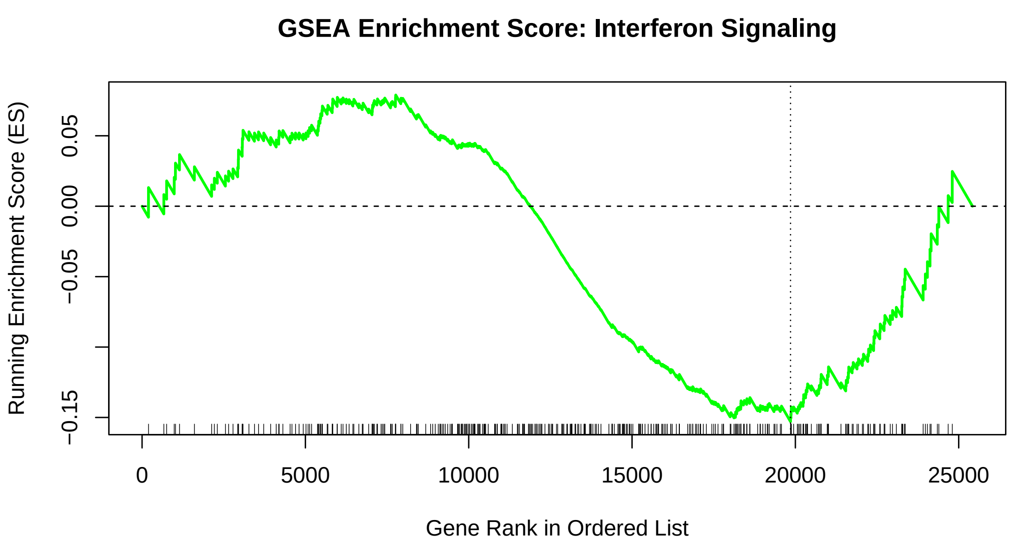
```r
plot_gsea_es(
  ranked_stats  = ranked_stats,
  gene_set      = gene_sets[["heat_shock_proteostasis"]],
  gene_set_name = gene_set_labels[["heat_shock_proteostasis"]]
)
```



Interferon Signaling

The interferon signaling set (374 genes) produces a small positive peak early in the list before a sustained decline, yielding a negative final ES (-0.153). This suggests that interferon pathway genes are not preferentially upregulated by WBH.

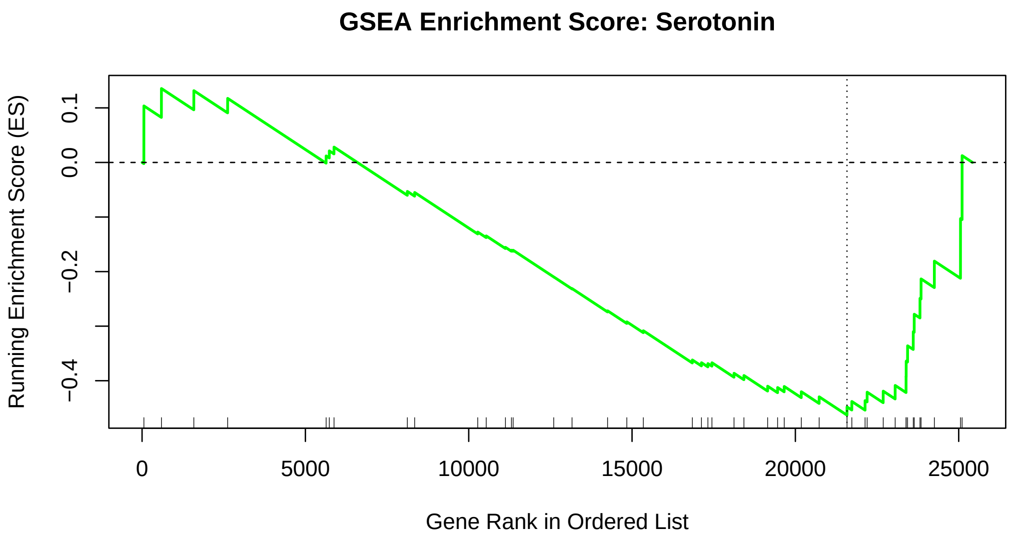
```
plot_gsea_es(
  ranked_stats  = ranked_stats,
  gene_set      = gene_sets[["interferon_signaling"]],
  gene_set_name = gene_set_labels[["interferon_signaling"]]
)
```



Serotonin

The serotonin gene set (46 genes) shows a very early, brief positive peak followed by a nearly linear decline through the remainder of the list, resulting in an ES of -0.463. The plot reflects the small set size and the presence serotonin genes across the lower half of the ranking.

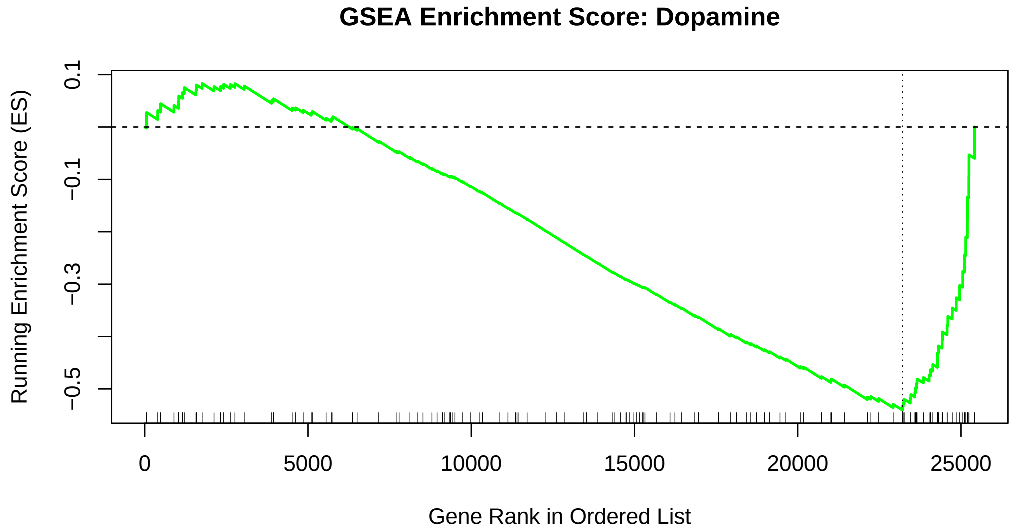
```r
plot_gsea_es(
  ranked_stats  = ranked_stats,
  gene_set      = gene_sets[["serotonin"]],
  gene_set_name = gene_set_labels[["serotonin"]]
)
```



Dopamine

Dopamine pathway genes (131 genes) show a pattern similar to serotonin: a small positive early peak followed by a long linear decline, giving the largest negative ES of the four sets (-0.541). Taken together with the serotonin result, this suggests that neurotransmitter signaling genes tend to be downregulated, or at least not upregulated under WBH in this dataset.

```r
plot_gsea_es(
  ranked_stats  = ranked_stats,
  gene_set      = gene_sets[["dopamine"]],
  gene_set_name = gene_set_labels[["dopamine"]]
)
```



Reproduce GSEA using Broad Institute Software using DSeq2

We can also recreate GSEA enrichment plots using the software app rather than running the manual pipeline. Using the same starting counts matrix, we perform data cleaning, id mapping, and low-expression filtering in an analogous manner to above. We also include sex as a confounding variable in the design matrix and pre-rank the genes using (sign of Fold Change * -log10 of P-value) score in decreasing order. We've also removed reads that could not be mapped to a gene symbol since these typically represent on-coding RNAs, pseudogenes, or poorly characterized regions that may clutter our data. A ranked file is written (DE_WBH_Sex_Adjusted.rnk).

```
cnt <- read.csv("wbh_counts_matrix.csv", row.names = 1, check.names = FALSE)
cnt <- as.matrix(cnt)

# Clean sample names and Ensembl versions
colnames(cnt) <- sub("\\.Aligned\\.sortedByCoord\\.out\\.bam$", "", basename(colnames(cnt)))
colnames(cnt) <- sub("\\.ReadsPerGene\\.out\\.tab$", "", colnames(cnt))
ensembl_ids <- sub("\\..*$", "", rownames(cnt))

# Annotate with Symbols
gene_annot <- AnnotationDbi::select(org.Hs.eg.db, keys = ensembl_ids,
                                    columns = c("SYMBOL"), keytype = "ENSEMBL")
gene_symbols <- gene_annot$SYMBOL[match(ensembl_ids, gene_annot$ENSEMBL)]
gene_symbols[is.na(gene_symbols)] <- ensembl_ids[is.na(gene_symbols)]
gene_symbols <- make.unique(gene_symbols)
rownames(cnt) <- gene_symbols

# Filtering
cpm_vals <- edgeR::cpm(cnt)
keep <- rowSums(cpm_vals > 1) >= 2
cnt_filtered <- cnt[keep, ]

# Only keep genes that were successfully renamed to Symbols
cnt_final <- cnt_filtered[!grepl("^ENSG", rownames(cnt_filtered)), ]

# Define Metadata (19 Samples)
```

```r
sample_info <- tibble(
  sample = c("SRR34413163", "SRR34413164", "SRR34413165", "SRR34413166", "SRR34413167",
             "SRR34413168", "SRR34413169", "SRR34413170", "SRR34413171", "SRR34413172",
             "SRR34413174", "SRR34413175", "SRR34413176", "SRR34413177", "SRR34413178",
             "SRR34413179", "SRR34413180", "SRR34413181", "SRR34413182"),
  group = factor(c(rep("WBH", 10), rep("Sham", 9)), levels = c("Sham", "WBH")),
  sex = factor(c("M", "M", "M", "F", "F", "F", "F", "F", "F", "F",
                 "M", "M", "M", "M", "M", "F", "F", "F", "F"))
) %>% as.data.frame()

rownames(sample_info) <- sample_info$sample
cnt_final <- cnt_final[, sample_info$sample] # Ensure columns match metadata

# Run DESeq2
dds <- DESeqDataSetFromMatrix(countData = round(cnt_final), # Round in case of decimals
                              colData = sample_info,
                              design = ~ sex + group)

dds <- DESeq(dds)

# Get results, utilizes paper's alpha (FDR < 0.01)
res <- results(dds, contrast=c("group", "WBH", "Sham"), alpha = 0.01)

# Rank for GSEA
res_df <- as.data.frame(res) %>%
  rownames_to_column("Gene") %>%
  # Rank Score = sign of Fold Change * -log10 of P-value
  mutate(rank_score = sign(log2FoldChange) * -log10(pvalue)) %>%
  arrange(desc(rank_score))

# Save file
write.table(res_df[, c("Gene", "rank_score")],
            "DE_WBH_Sex_Adjusted.rnk",
            sep = "\t", row.names = FALSE, col.names = FALSE, quote = FALSE)

# Check for specific heat-shock genes
res_df %>% filter(Gene %in% c("HSPA1A", "HSPA1B", "HSPA6", "DNAJB1"))
```

```
##     Gene   baseMean log2FoldChange     lfcSE      stat     pvalue      padj
## 1 HSPA1A  148.49398      0.5875963 0.2553274 2.3013442 0.02137218 0.7704964
## 2 HSPA1B   86.22205      0.5637485 0.2671522 2.1102149 0.03483985 0.7930173
## 3 DNAJB1  641.92439      0.1920264 0.1349267 1.4231907 0.15468087 0.8607733
## 4  HSPA6 1701.41458      0.1383102 0.1982720 0.6975781 0.48544111 0.9303177
##   rank_score
## 1  1.6701511
## 2  1.4579237
## 3  0.8105634
## 4  0.3138634
```

```r
# Check what our top 5 DE genes are
sum(res_df$padj < 0.05, na.rm = TRUE)
```

```
## [1] 5
```

```r
top_5_genes <- res_df %>%
  filter(padj < 0.05) %>%
  arrange(padj) %>%
  dplyr::select(Gene, baseMean, log2FoldChange, pvalue, padj)

print(top_5_genes)
```

```
##    Gene   baseMean log2FoldChange       pvalue        padj
## 1 DNM1L  741.58136     -0.2708330 5.728004e-07 0.004962170
## 2 ATG2B 1755.81451     -0.2817325 5.198222e-07 0.004962170
## 3 FKBP4  386.27673      0.8221959 1.710445e-06 0.009878393
## 4 VEPH1   95.35925     -0.8979053 5.883870e-06 0.025485985
## 5 MDGA1  139.42071      3.5392068 1.205477e-05 0.041772174
```

Reproduce GSEA using Broad Institute Software using EdgeR

Both DESeq2 and EdgeR use negative binomial distributions to model RNA-sequencing data, but they differ in how they normalize data and handle dispersion. EdgeR uses Trimmed Mean of M-values that trims the most high and low expressed genes. In comparison, DESeq2 uses Median of Ratios which scales libraries using a pseudo-reference gene for normalization. For dispersion, EdgeR uses the Quasi-Likelihood (QL) F-test which is much more conservative than DESeq2's Wald Test methodology in controlling false positive. However, unlike DESeq2, EdgeR does not have a method of managing outliers. Overall, EdgeR is better at sensitivity for low-expresion genes and is preferred for small sample sizes. This code will produce a ranked file called (edgeR_WBH_Sex_Adjusted.rnk).

```r
design <- model.matrix(~ sex + group, data = sample_info)

# Run edgeR using your existing cnt_final
dge <- DGEList(counts = cnt_final)
dge <- calcNormFactors(dge)
dge <- estimateDisp(dge, design)
fit <- glmQLFit(dge, design)
qlf <- glmQLFTest(fit, coef = "groupWBH")

# Create the .rnk file
res_edger <- as.data.frame(topTags(qlf, n = Inf)) %>%
  rownames_to_column("Gene") %>%
  mutate(rank_score = sign(logFC) * -log10(PValue)) %>%
  arrange(desc(rank_score))

write.table(res_edger[, c("Gene", "rank_score")],
            "edgeR_WBH_Sex_Adjusted.rnk",
            sep = "\t", row.names = FALSE, col.names = FALSE, quote = FALSE)

# Check for specific heat-shock genes
res_edger %>% filter(Gene %in% c("HSPA1A", "HSPA1B", "HSPA6", "DNAJB1"))
```

```
##     Gene     logFC   logCPM         F     PValue       FDR rank_score
## 1 HSPA1A 0.5825024 3.336822 5.8062385 0.02624867 0.7363851  1.5808927
## 2 HSPA1B 0.5543217 2.565681 4.8620390 0.04010428 0.7363851  1.3968093
## 3 DNAJB1 0.1914910 5.431045 2.0915474 0.16428507 0.7363851  0.7844019
## 4  HSPA6 0.1414970 6.835561 0.5498694 0.46736618 0.8098574  0.3303427
```

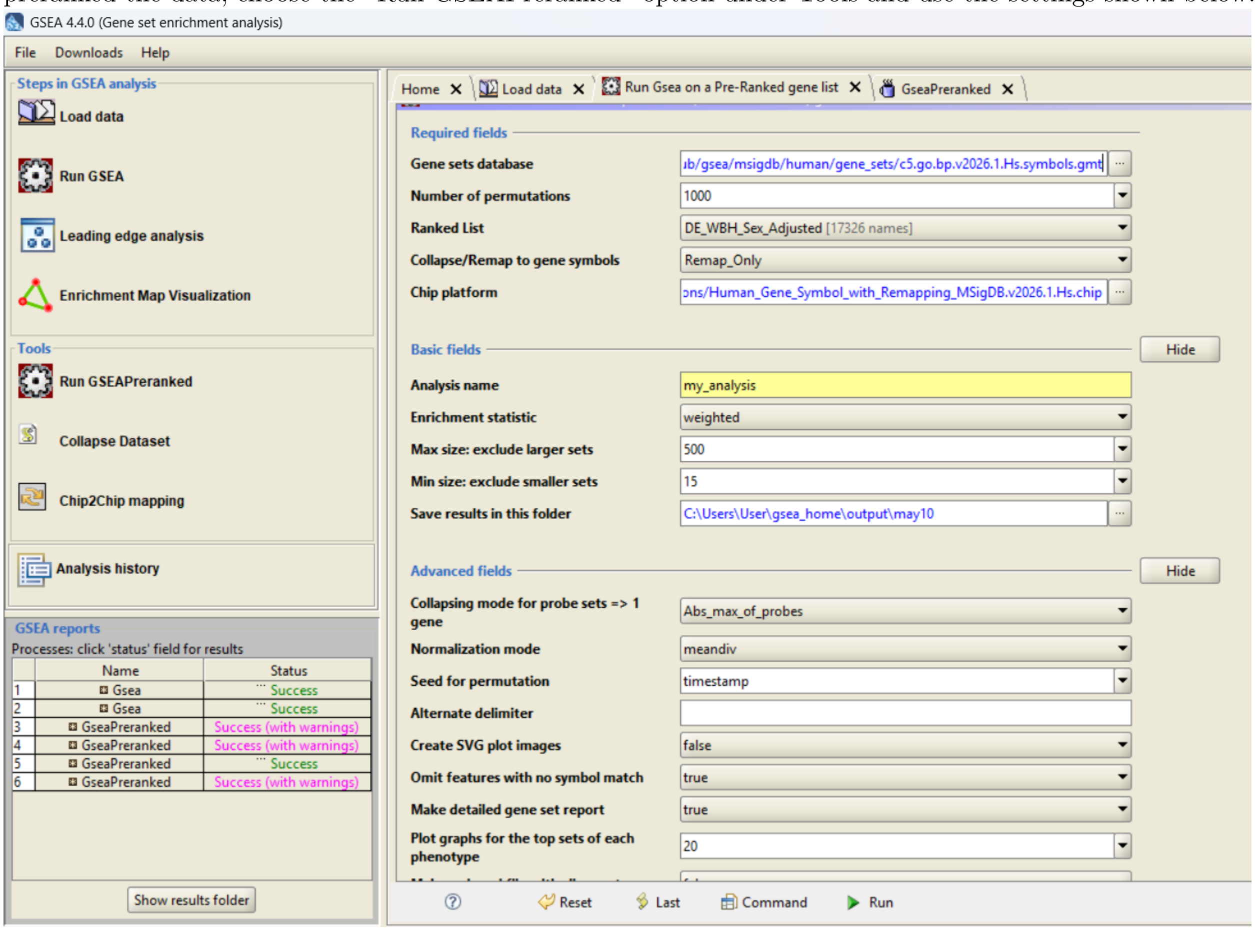
```r
# Check what our top 5 DE genes are
sum(res_edger$FDR < 0.05, na.rm = TRUE)
```

```
## [1] 0
```

```r
top_5_genes <- res_edger %>%
  filter(FDR < 0.05) %>%
  arrange(FDR) %>%
  dplyr::select(Gene, logCPM, logFC, PValue, FDR)

print(top_5_genes)
```

```
## [1] Gene   logCPM logFC  PValue FDR
## <0 rows> (or 0-length row.names)
```

In the GSEA app, load the ranked files using the load data tab in the left side of the screen. Since we have preranked the data, choose the “Run GSEAPreranked” option under Tools and use the settings shown below.



Since we want to investigate molecular mechanisms in additional to biological pathways, we will use 3 different gene datasets including: ftp.broadinstitute.org://pub/gsea/msigdb/human/gene_sets/h.all.v2026.1.Hs.symbols.gmt ftp.broadinstitute.org://pub/gsea/msigdb/human/gene_sets/c2.cp.reactome.v2026.1.Hs.symbols.gmt ftp.broadinstitute.org://pub/gsea/msigdb/human/gene_sets/c5.go.bp.v2026.1.Hs.symbols.gmt

After running the analysis, these were the following results obtained for the DESeq2 preranked list and EdgeR preranked lists respectively:

While our GSEA results don’t exactly match the paper’s (likely due to differences in raw gene counts, sample subset that was used, differences in mapping/annotation methodology, removal of outliers, and other aspects mentioned in our presentation), we can see that our GSEA results show upregulation in heat shock and neuronal pathways in the WBH condition for both DESeq and EdgeR respectively. This matches the expected biological context that the experiment was conducted in (pathways that are relevant to MDD and exposure

Enrichment in phenotype: **na**

- 1968 / 4735 gene sets are upregulated in phenotype **na_pos**
- 9 gene sets are significant at FDR < 25%
- 36 gene sets are significantly enriched at nominal pvalue < 1%
- 132 gene sets are significantly enriched at nominal pvalue < 5%
- Snapshot of enrichment results
- Detailed enrichment results in html format
- Detailed enrichment results in TSV format (tab delimited text)
- Guide to interpret results

Enrichment in phenotype: **na**

- 2767 / 4735 gene sets are upregulated in phenotype **na_neg**
- 67 gene sets are significantly enriched at FDR < 25%
- 157 gene sets are significantly enriched at nominal pvalue < 1%
- 392 gene sets are significantly enriched at nominal pvalue < 5%
- Snapshot of enrichment results
- Detailed enrichment results in html format
- Detailed enrichment results in TSV format (tab delimited text)
- Guide to interpret results

Figure 1: DESeq2 Results

	GS follow link to MSigDB	GS DETAILS	SIZE	ES	NES	NOM p-val	FDR q-val	FWER p-val
1	REACTOME_ATTENUATION_PHASE	Details ...	24	0.77	2.21	0.000	0.001	0.002
2	REACTOME_HSF1_DEPENDENT_TRANSACTIVATION	Details ...	30	0.71	2.16	0.000	0.002	0.005
3	GOBP_PROTEIN_REFOLDING	Details ...	26	0.66	1.94	0.000	0.106	0.349
4	REACTOME_HSP90_CHAPERONE_CYCLE_FOR_STEROID_HORMONE_RECEPTORS_SHR_IN_THE_PRESENCE_OF_LIGAND	Details ...	49	0.57	1.91	0.000	0.130	0.500
5	GOBP_POSITIVE_REGULATION_OF_SYNAPTIC_TRANSMISSION_GLUTAMATERGIC	Details ...	16	0.72	1.91	0.002	0.111	0.524
6	REACTOME_HSF1_ACTIVATION	Details ...	27	0.65	1.90	0.000	0.102	0.554
7	REACTOME_MITOCHONDRIAL_UNFOLDED_PROTEIN_RESPONSE_UPRMT	Details ...	17	0.70	1.87	0.000	0.143	0.728
8	REACTOME_CD22_MEDIATED_BCR_REGULATION	Details ...	49	0.55	1.85	0.002	0.159	0.819
9	REACTOME_ANTIGEN_ACTIVATES_B_CELL_RECEPTOR_BCR_LEADING_TO_GENERATION_OF_SECOND_MESSENGERS	Details ...	74	0.51	1.84	0.000	0.153	0.842
10	GOBP_NEGATIVE_REGULATION_OF_ENDOPLASMIC_RETICULUM_STRESS_INDUCED_INTRINSIC_APOPTOTIC_SIGNALING_PATHWAY	Details ...	18	0.65	1.78	0.002	0.303	0.983

Figure 2: DESeq2 Upregulated

Enrichment in phenotype: na

- 3108 / 4996 gene sets are upregulated in phenotype **na_pos**
- 357 gene sets are significant at FDR < 25%
- 246 gene sets are significantly enriched at nominal pvalue < 1%
- 502 gene sets are significantly enriched at nominal pvalue < 5%
- Snapshot of enrichment results
- Detailed enrichment results in html format
- Detailed enrichment results in TSV format (tab delimited text)
- Guide to interpret results

Enrichment in phenotype: na

- 1888 / 4996 gene sets are upregulated in phenotype **na_neg**
- 335 gene sets are significantly enriched at FDR < 25%
- 244 gene sets are significantly enriched at nominal pvalue < 1%
- 474 gene sets are significantly enriched at nominal pvalue < 5%
- Snapshot of enrichment results
- Detailed enrichment results in html format
- Detailed enrichment results in TSV format (tab delimited text)
- Guide to interpret results

Figure 3: EdgeR Results

	GS follow link to MSigDB	GS DETAILS	SIZE	ES	NES	NOM p-val	FDR q-val	FWER p-val
1	GOBP_POSITIVE_REGULATION_OF_SYNAPTIC_TRANSMISSION_GLUTAMATERGIC	Details ...	26	0.72	2.19	0.000	0.002	0.002
2	REACTOME_ATTENUATION_PHASE	Details ...	24	0.73	2.16	0.000	0.001	0.003
3	GOBP_REGULATION_OF_GLIAL_CELL_DIFFERENTIATION	Details ...	62	0.57	2.03	0.000	0.012	0.046
4	REACTOME_POTASSIUM_CHANNELS	Details ...	84	0.55	2.03	0.000	0.009	0.047
5	GOBP_NEURON_FATE_COMMITMENT	Details ...	48	0.59	2.01	0.000	0.011	0.070
6	REACTOME_NEUREXINS_AND_NEUROLIGINS	Details ...	50	0.58	1.99	0.000	0.014	0.107
7	REACTOME_CD22_MEDIATED_BCR_REGULATION	Details ...	49	0.57	1.97	0.000	0.018	0.151
8	GOBP_NEUROMUSCULAR_PROCESS_CONTROLLING_BALANCE	Details ...	51	0.57	1.97	0.000	0.016	0.155
9	REACTOME_PROTEIN_PROTEIN_INTERACTIONS_AT_SYNAPSES	Details ...	71	0.54	1.96	0.000	0.016	0.174
10	GOBP_REGULATION_OF_SYNAPTIC_TRANSMISSION_GLUTAMATERGIC	Details ...	61	0.55	1.95	0.000	0.015	0.179

Figure 4: EdgeR Upregulated

to high heat). The EdgeR results were a bit more sensitive and significant comparative to the DESeq2 results, possibly because the strength of our biological signal and low sample size made this methodology a bit more robust given our conditions.

Clicking on individual pathways shows the gene enrichment plots. The peaks are consistently sharp and occur at the leftmost side of the graph. Genes with lower NES and significance show more rounded peaks that are present closer to the middle.

Overall, although we cannot make conclusive statements regarding WBH treatment and the enrichment of serotonin/dopamine transport pathways, our GSEA results still align with the overall broader trends within our expectations, with upregulation of pathways regarding neuronal activity and synaptic transmission which are implicated in depression studies as well as heat shock protein production and protein folding.

Session info

```
sessionInfo()
```

```
## R version 4.5.1 (2025-06-13)
## Platform: aarch64-apple-darwin20
## Running under: macOS Sequoia 15.6.1
##
## Matrix products: default
## BLAS:   /Library/Frameworks/R.framework/Versions/4.5-arm64/Resources/lib/libRblas.0.dylib
## LAPACK: /Library/Frameworks/R.framework/Versions/4.5-arm64/Resources/lib/libRlapack.dylib;  LAPACK ve
##
## locale:
## [1] en_US.UTF-8/en_US.UTF-8/en_US.UTF-8/C/en_US.UTF-8/en_US.UTF-8
##
## time zone: America/New_York
## tzcode source: internal
##
## attached base packages:
## [1] stats4    stats     graphics  grDevices utils     datasets  methods
## [8] base
##
## other attached packages:
##  [1] gt_1.3.0                    lubridate_1.9.5
##  [3] forcats_1.0.1               stringr_1.6.0
##  [5] dplyr_1.2.1                 purrr_1.2.2
##  [7] readr_2.2.0                 tidyr_1.3.2
##  [9] tibble_3.3.1                ggplot2_4.0.3
## [11] tidyverse_2.0.0             DESeq2_1.50.2
## [13] SummarizedExperiment_1.40.0 MatrixGenerics_1.22.0
## [15] matrixStats_1.5.0           GenomicRanges_1.62.1
## [17] Seqinfo_1.0.0               edgeR_4.8.2
## [19] limma_3.66.0                GO.db_3.22.0
## [21] org.Hs.eg.db_3.22.0         AnnotationDbi_1.72.0
## [23] IRanges_2.44.0              S4Vectors_0.48.1
## [25] Biobase_2.70.0              BiocGenerics_0.56.0
## [27] generics_0.1.4
##
## loaded via a namespace (and not attached):
##  [1] tidyselect_1.2.1    farver_2.1.2        blob_1.3.0
##  [4] Biostrings_2.78.0   S7_0.2.2            fastmap_1.2.0
##  [7] digest_0.6.39       timechange_0.4.0    lifecycle_1.0.5
```

```
## [10] statmod_1.5.1       KEGGREST_1.50.0     RSQLite_2.4.6
## [13] magrittr_2.0.5      compiler_4.5.1      rlang_1.2.0
## [16] tools_4.5.1         utf8_1.2.6          yaml_2.3.12
## [19] knitr_1.51          labeling_0.4.3      S4Arrays_1.10.1
## [22] bit_4.6.0           DelayedArray_0.36.1 xml2_1.5.2
## [25] RColorBrewer_1.1-3  abind_1.4-8         BiocParallel_1.44.0
## [28] withr_3.0.2         grid_4.5.1          scales_1.4.0
## [31] cli_3.6.6           rmarkdown_2.31      crayon_1.5.3
## [34] otel_0.2.0          rstudioapi_0.18.0   httr_1.4.8
## [37] tzdb_0.5.0          DBI_1.3.0           cachem_1.1.0
## [40] parallel_4.5.1      XVector_0.50.0      vctrs_0.7.3
## [43] Matrix_1.7-5        hms_1.1.4           bit64_4.8.0
## [46] locfit_1.5-9.12     glue_1.8.1          codetools_0.2-20
## [49] stringi_1.8.7       gtable_0.3.6        pillar_1.11.1
## [52] htmltools_0.5.9     R6_2.6.1            evaluate_1.0.5
## [55] lattice_0.22-9      png_0.1-9           memoise_2.0.1
## [58] Rcpp_1.1.1-1.1      SparseArray_1.10.10 xfun_0.57
## [61] fs_2.1.0            pkgconfig_2.0.3
```